dl-lab-6

April 18, 2025

1 Lab on different optimizers

Dataset: The network will be trained and tested using the MNIST dataset of handwritten digits (0–9). It can be downloaded directly from Keras/Pytorch/TensorFlow, e.g- (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

Data Splitting: Randomly divide the dataset into train, validation, and test splits in a 60:20:20 ratio. **Network Architecture**: Two hidden layers, each with 200 neurons (you may change it according to your need) - Use a learning rate of 0.01 - Use sigmoid as the activation function for hidden layers - Use softmax as the activation function for the output layer

Optimizer Comparison: Modify the experiment to use different optimizers and compare their performance: - Stochastic Gradient Descent (SGD) - Adam - RMSprop - Momentum-based Gradient Descent

Note- Weights(parameters) initialization should be same for every optimizer. Randomly initialize all the parameters once and store. Use these parameters for all different optimizer.

Evaluation: Include plots of training and validation loss vs. epochs for each case Report the test accuracy for each architecture and optimizer Analyze and compare the impact of different optimizers on training speed and final accuracy

[1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import random_split, DataLoader
from torchvision import datasets, transforms
import copy
import matplotlib.pyplot as plt

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Reproducibility
seed = 42
torch.manual_seed(seed)
if device.type == 'cuda':
    torch.cuda.manual_seed_all(seed)
```

1.1 Load and Split MNIST

[2]:
```
# Transform: flatten + normalize to [0,1]
transform = transforms.Compose([
    transforms.ToTensor(),            # [0,255]→[0,1]
    transforms.Normalize((0.5,), (0.5,))  # optional centering
])

# Download full training set (60k examples)
full_train = datasets.MNIST(root='.', train=True, download=True,␣
 ↪transform=transform)
test_set  = datasets.MNIST(root='.', train=False, download=True,␣
 ↪transform=transform)

# Split full_train → train (36k), val (12k), remaining 12k to match 60:20:20
n_train = int(0.6 * len(full_train))   # 36000
n_val   = int(0.2 * len(full_train))   # 12000
n_unused= len(full_train) - n_train - n_val  # 12000 (we could discard or merge)
train_set, val_set, _ = random_split(full_train, [n_train, n_val, n_unused],
                                     generator=torch.Generator().
 ↪manual_seed(seed))

batch_size = 128
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_set,   batch_size=batch_size, shuffle=False)
test_loader  = DataLoader(test_set,  batch_size=batch_size, shuffle=False)
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-images-
idx3-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-images-
idx3-ubyte.gz to ./MNIST/raw/train-images-idx3-ubyte.gz
```

```
100%|      | 9.91M/9.91M [00:00<00:00, 16.3MB/s]
```

```
Extracting ./MNIST/raw/train-images-idx3-ubyte.gz to ./MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-labels-
idx1-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-labels-
idx1-ubyte.gz to ./MNIST/raw/train-labels-idx1-ubyte.gz
```

```
100%|      | 28.9k/28.9k [00:00<00:00, 476kB/s]

Extracting ./MNIST/raw/train-labels-idx1-ubyte.gz to ./MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-
idx3-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-
idx3-ubyte.gz to ./MNIST/raw/t10k-images-idx3-ubyte.gz

100%|      | 1.65M/1.65M [00:00<00:00, 4.52MB/s]

Extracting ./MNIST/raw/t10k-images-idx3-ubyte.gz to ./MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-
idx1-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-
idx1-ubyte.gz to ./MNIST/raw/t10k-labels-idx1-ubyte.gz

100%|      | 4.54k/4.54k [00:00<00:00, 7.40MB/s]

Extracting ./MNIST/raw/t10k-labels-idx1-ubyte.gz to ./MNIST/raw
```

1.2 Define MLP and Initialize weights

2 hidden layers with 200 neurons + sigmoid activations

Grab one copy of the randomly initialized parameters (initial_state) to reload before each optimizer run

[3]:
```python
class MLP(nn.Module):
    def __init__(self, input_dim=28*28, hidden_dim=200, output_dim=10):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
        self.sig  = nn.Sigmoid()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = x.view(x.size(0), -1)
```

```
        x = self.sig(self.fc1(x))
        x = self.sig(self.fc2(x))
        logits = self.fc3(x)
        # Note: we return raw logits for training with CrossEntropyLoss,
        # but you can apply self.softmax(logits) at inference.
        return logits

# Instantiate and capture initial weights
base_model = MLP().to(device)
initial_state = copy.deepcopy(base_model.state_dict())
```

1.3 Traning and Validation

Using CrossEntropyLoss, which internally applies log_softmax + NLL, so we feed it raw logits.

[4]:
```
def train_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss = 0.0
    for X, y in loader:
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, y)            # CrossEntropyLoss
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * X.size(0)
    return running_loss / len(loader.dataset)

def eval_epoch(model, loader, criterion):
    model.eval()
    running_loss, correct = 0.0, 0
    with torch.no_grad():
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            loss = criterion(logits, y)
            running_loss += loss.item() * X.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
    avg_loss = running_loss / len(loader.dataset)
    acc = correct / len(loader.dataset)
    return avg_loss, acc
```

1.4 Run experiments for each optimizer

Loop over each optimizer, reload the same initial parameters, train for 20 epochs, record losses, and finally measure test set accuracy

[5]:
```
optimizers = {
    'SGD':      lambda params: optim.SGD(params,      lr=0.01),
    'SGD+Mom':  lambda params: optim.SGD(params,      lr=0.01, momentum=0.9),
    'RMSprop':  lambda params: optim.RMSprop(params,  lr=0.01),
    'Adam':     lambda params: optim.Adam(params,     lr=0.01),
}

num_epochs = 20
results = {}

for name, opt_fn in optimizers.items():
    # Reload initial weights
    model = MLP().to(device)
    model.load_state_dict(initial_state)
    optimizer = opt_fn(model.parameters())
    criterion = nn.CrossEntropyLoss()

    train_losses, val_losses = [], []
    for epoch in range(1, num_epochs+1):
        tr_loss = train_epoch(model, train_loader, criterion, optimizer)
        va_loss, _  = eval_epoch(model, val_loader,   criterion)
        train_losses.append(tr_loss)
        val_losses.append(va_loss)
        print(f"[{name}] Epoch {epoch}/{num_epochs} - train_loss: {tr_loss:.
 ↪4f}, val_loss: {va_loss:.4f}")

    # Final test accuracy
    _, test_acc = eval_epoch(model, test_loader, criterion)

    results[name] = {
        'train_losses': train_losses,
        'val_losses':   val_losses,
        'test_acc':     test_acc
    }
    print(f"→ {name} test accuracy: {test_acc*100:.2f}%\n")
```

```
The history saving thread hit an unexpected error (OperationalError('attempt to
write a readonly database')).History will not be written to the database.
[SGD] Epoch 1/20 - train_loss: 2.2998, val_loss: 2.2941
[SGD] Epoch 2/20 - train_loss: 2.2911, val_loss: 2.2861
[SGD] Epoch 3/20 - train_loss: 2.2826, val_loss: 2.2781
[SGD] Epoch 4/20 - train_loss: 2.2717, val_loss: 2.2655
[SGD] Epoch 5/20 - train_loss: 2.2573, val_loss: 2.2489
[SGD] Epoch 6/20 - train_loss: 2.2373, val_loss: 2.2238
[SGD] Epoch 7/20 - train_loss: 2.2076, val_loss: 2.1886
[SGD] Epoch 8/20 - train_loss: 2.1633, val_loss: 2.1346
[SGD] Epoch 9/20 - train_loss: 2.0965, val_loss: 2.0540
[SGD] Epoch 10/20 - train_loss: 2.0001, val_loss: 1.9405
```

```
[SGD] Epoch 11/20 - train_loss: 1.8735, val_loss: 1.8027
[SGD] Epoch 12/20 - train_loss: 1.7269, val_loss: 1.6499
[SGD] Epoch 13/20 - train_loss: 1.5758, val_loss: 1.5014
[SGD] Epoch 14/20 - train_loss: 1.4343, val_loss: 1.3665
[SGD] Epoch 15/20 - train_loss: 1.3090, val_loss: 1.2496
[SGD] Epoch 16/20 - train_loss: 1.2010, val_loss: 1.1491
[SGD] Epoch 17/20 - train_loss: 1.1084, val_loss: 1.0649
[SGD] Epoch 18/20 - train_loss: 1.0295, val_loss: 0.9902
[SGD] Epoch 19/20 - train_loss: 0.9615, val_loss: 0.9277
[SGD] Epoch 20/20 - train_loss: 0.9030, val_loss: 0.8726
→ SGD test accuracy: 76.56%

[SGD+Mom] Epoch 1/20 - train_loss: 2.2440, val_loss: 2.0486
[SGD+Mom] Epoch 2/20 - train_loss: 1.4317, val_loss: 0.9348
[SGD+Mom] Epoch 3/20 - train_loss: 0.7315, val_loss: 0.5903
[SGD+Mom] Epoch 4/20 - train_loss: 0.5178, val_loss: 0.4652
[SGD+Mom] Epoch 5/20 - train_loss: 0.4321, val_loss: 0.4117
[SGD+Mom] Epoch 6/20 - train_loss: 0.3858, val_loss: 0.3800
[SGD+Mom] Epoch 7/20 - train_loss: 0.3541, val_loss: 0.3552
[SGD+Mom] Epoch 8/20 - train_loss: 0.3303, val_loss: 0.3335
[SGD+Mom] Epoch 9/20 - train_loss: 0.3106, val_loss: 0.3147
[SGD+Mom] Epoch 10/20 - train_loss: 0.2938, val_loss: 0.3007
[SGD+Mom] Epoch 11/20 - train_loss: 0.2782, val_loss: 0.2818
[SGD+Mom] Epoch 12/20 - train_loss: 0.2636, val_loss: 0.2739
[SGD+Mom] Epoch 13/20 - train_loss: 0.2505, val_loss: 0.2596
[SGD+Mom] Epoch 14/20 - train_loss: 0.2381, val_loss: 0.2464
[SGD+Mom] Epoch 15/20 - train_loss: 0.2263, val_loss: 0.2398
[SGD+Mom] Epoch 16/20 - train_loss: 0.2160, val_loss: 0.2253
[SGD+Mom] Epoch 17/20 - train_loss: 0.2060, val_loss: 0.2164
[SGD+Mom] Epoch 18/20 - train_loss: 0.1964, val_loss: 0.2100
[SGD+Mom] Epoch 19/20 - train_loss: 0.1867, val_loss: 0.2022
[SGD+Mom] Epoch 20/20 - train_loss: 0.1791, val_loss: 0.1953
→ SGD+Mom test accuracy: 94.60%

[RMSprop] Epoch 1/20 - train_loss: 2.4817, val_loss: 2.5537
[RMSprop] Epoch 2/20 - train_loss: 2.3517, val_loss: 2.4139
[RMSprop] Epoch 3/20 - train_loss: 2.3314, val_loss: 2.4447
[RMSprop] Epoch 4/20 - train_loss: 2.3249, val_loss: 2.4416
[RMSprop] Epoch 5/20 - train_loss: 2.3220, val_loss: 2.3445
[RMSprop] Epoch 6/20 - train_loss: 2.3184, val_loss: 2.3681
[RMSprop] Epoch 7/20 - train_loss: 2.3161, val_loss: 2.3427
[RMSprop] Epoch 8/20 - train_loss: 2.3111, val_loss: 2.3548
[RMSprop] Epoch 9/20 - train_loss: 2.3106, val_loss: 2.3246
[RMSprop] Epoch 10/20 - train_loss: 2.3079, val_loss: 2.3161
[RMSprop] Epoch 11/20 - train_loss: 2.3067, val_loss: 2.3152
[RMSprop] Epoch 12/20 - train_loss: 2.3057, val_loss: 2.3038
[RMSprop] Epoch 13/20 - train_loss: 2.3055, val_loss: 2.3076
[RMSprop] Epoch 14/20 - train_loss: 2.3052, val_loss: 2.3045
```

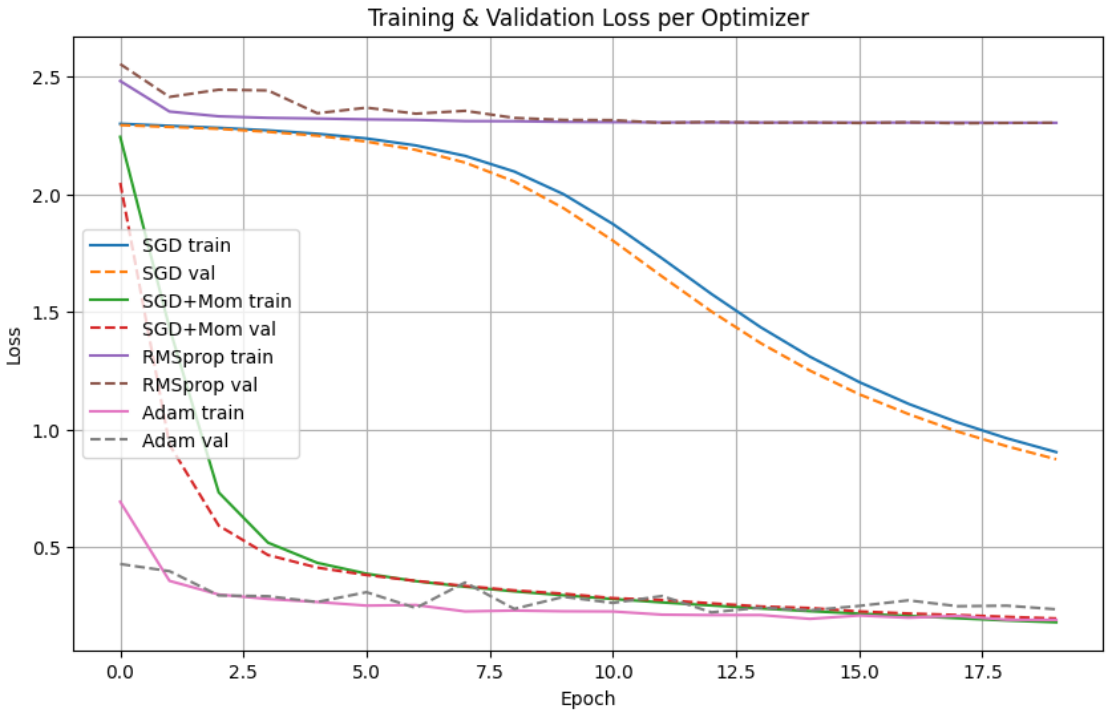
```
[RMSprop] Epoch 15/20 - train_loss: 2.3048, val_loss: 2.3054
[RMSprop] Epoch 16/20 - train_loss: 2.3050, val_loss: 2.3030
[RMSprop] Epoch 17/20 - train_loss: 2.3050, val_loss: 2.3063
[RMSprop] Epoch 18/20 - train_loss: 2.3050, val_loss: 2.3018
[RMSprop] Epoch 19/20 - train_loss: 2.3041, val_loss: 2.3032
[RMSprop] Epoch 20/20 - train_loss: 2.3039, val_loss: 2.3044
→ RMSprop test accuracy: 9.82%

[Adam] Epoch 1/20 - train_loss: 0.6920, val_loss: 0.4269
[Adam] Epoch 2/20 - train_loss: 0.3553, val_loss: 0.3966
[Adam] Epoch 3/20 - train_loss: 0.2972, val_loss: 0.2933
[Adam] Epoch 4/20 - train_loss: 0.2781, val_loss: 0.2900
[Adam] Epoch 5/20 - train_loss: 0.2654, val_loss: 0.2654
[Adam] Epoch 6/20 - train_loss: 0.2500, val_loss: 0.3071
[Adam] Epoch 7/20 - train_loss: 0.2526, val_loss: 0.2399
[Adam] Epoch 8/20 - train_loss: 0.2252, val_loss: 0.3484
[Adam] Epoch 9/20 - train_loss: 0.2290, val_loss: 0.2361
[Adam] Epoch 10/20 - train_loss: 0.2258, val_loss: 0.2875
[Adam] Epoch 11/20 - train_loss: 0.2251, val_loss: 0.2614
[Adam] Epoch 12/20 - train_loss: 0.2115, val_loss: 0.2909
[Adam] Epoch 13/20 - train_loss: 0.2096, val_loss: 0.2214
[Adam] Epoch 14/20 - train_loss: 0.2099, val_loss: 0.2414
[Adam] Epoch 15/20 - train_loss: 0.1936, val_loss: 0.2299
[Adam] Epoch 16/20 - train_loss: 0.2080, val_loss: 0.2487
[Adam] Epoch 17/20 - train_loss: 0.1984, val_loss: 0.2724
[Adam] Epoch 18/20 - train_loss: 0.2062, val_loss: 0.2474
[Adam] Epoch 19/20 - train_loss: 0.1898, val_loss: 0.2496
[Adam] Epoch 20/20 - train_loss: 0.1892, val_loss: 0.2343
→ Adam test accuracy: 93.66%
```

1.5 Plotting loss curves

Overlay all four optimizers' curves to compare convergence speed and stability.

[6]:
```python
plt.figure(figsize=(10,6))
for name, res in results.items():
    plt.plot(res['train_losses'], label=f"{name} train")
    plt.plot(res['val_losses'],   label=f"{name} val", linestyle='--')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training & Validation Loss per Optimizer")
plt.legend()
plt.grid(True)
plt.show()
```



1.6 Reposting and Analysis

[7]:
```
for name, res in results.items():
    print(f"{name:10s} - Test Accuracy: {res['test_acc']*100:.2f}%")
```

```
SGD        - Test Accuracy: 76.56%
SGD+Mom    - Test Accuracy: 94.60%
RMSprop    - Test Accuracy: 9.82%
Adam       - Test Accuracy: 93.66%
```